



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE MORELIA

DEPARTAMENTO DE SISTEMAS COMPUTACIONALES

Lenguajes y Autómatas II

Grupo: C

Editor de Código y Traductor de Lenguajes de Programación de Alto Nivel

FECHA DE ENTREGA: 28 DE MAYO DE 2026

Profesora:

M.S.C. Alma Lilia Núñez González

Alumnos:

Ibarrola Origel Andrés Daniel

Romero García Emiliano

Parra Fernández Bruno Leonardo

Vidales Castillo Diego

1. Estructura General del Proyecto

El proyecto está desarrollado en Java con Maven y se divide en cuatro capas principales. La primera es el análisis léxico y sintáctico, implementado con ANTLR4 mediante la gramática EBDA.g4, que define los tokens y reglas del lenguaje personalizado. La segunda es el análisis semántico e interpretación, implementado en EBDACustomVisitor.java, que extiende EBDABaseVisitor<Object> y recorre el árbol sintáctico validando tipos, detectando errores semánticos y ejecutando las instrucciones. La tercera es la generación de código, implementada en cinco clases traductoras independientes (EBDAToPythonVisitor, EBDAToCVisitor, EBDAToCSharpVisitor, EBDAToJavaScriptVisitor, EBDAToCppVisitor), cada una extiende EBDABaseVisitor<String> y produce código equivalente en el lenguaje destino. La cuarta es la interfaz gráfica, implementada con JavaFX en EBDAEditor.java, que integra todas las capas anteriores en una aplicación visual.

2. Funcionamiento del Intérprete

El intérprete sigue el flujo estándar de un compilador. Primero el EBDALexer tokeniza el código fuente. Luego el EBDAParser construye el árbol sintáctico. Finalmente el EBDACustomVisitor recorre el árbol aplicando análisis semántico y ejecución simultánea.

La tabla de símbolos se implementa mediante HashMap<String, Simbolo> donde cada entrada almacena el tipo (healthpoints, chat, online) y el valor actual de la variable. Las validaciones semánticas incluyen: variable no declarada antes de usarse, redeclaración de variable con tipo diferente, tipos incompatibles en operaciones aritméticas, tipo incompatible en asignación, división entre cero, condición no booleana en rush y respawn, y uso incorrecto de buffear/nerfear sobre tipos no enteros.

3. Proceso de Traducción y Generación de Código

Cada traductor recorre el mismo árbol sintáctico que el intérprete pero en lugar de ejecutar produce un String con el código equivalente en el lenguaje destino. El mapeo de tipos es el siguiente:

EBDA	Python	C	C#	JavaScript	C++
healthpoints	int	int	int	let	int
chat	str	char[256]	string	let	string
online	bool	int	bool	let	bool

EBDA	Python	C	C#	JavaScript	C++
rush/afk	if/else	if/else	if/else	if/else	if/else
respawn	while	while	while	while	while
buffear	+= 1	++	++	++	++
nerfear	-= 1	--	--	--	--
ñ	print()	printf()	Console.WriteLine()	console.log()	cout